

NLP Milestone 2: Relation Extraction

Authors: Silvia Bonomi, Samuele D'Avenia, Andrea Nugara, Maria Pronestì

December 3, 2023

1 Introduction

For Milestone 2, the aim is to produce some baseline models for the Relation Extraction task. To be more specific, this report describes the results obtained by using both a Random Forest (RF), described in Section 3, and some results obtained using Deep Learning techniques 4.

Error analysis is performed on the RF tuned with the best hyper-parameters and on the most promising model from those analysed with deep learning techniques.

The comparison of the TACRED and TACREV datasets, along with using LLMs from the BERT family, are left as a task for the final submission.

2 Model Evaluation

For model evaluation the approach reported in Zhang et al. [2017] is used and is briefly described below. This is done to be able to compare results. Since the aim of the task is relation classification, we only consider as positive instances those where a relation is present. To be more specific the following are defined:

- True Positive (TP): occurs when the model correctly identifies a relation (does not apply if it correctly assigns no_relation).
- False Positive (FP): occurs when the model classifies the relation between two entities wrongly (but still identifies it as a relation).
- False Negative (FN): occurs when the model identifies some relation between the entities in the sentence, while the two are not related.

Following these definitions, the authors use micro-averaging precision, recall and F1 score to obtain the results reported in the paper.

The main advantage of this evaluation metric on datasets such as TACRED with a large number of sentences having *no_relation*, is that a model is not given a higher score for correctly predicting those.

One problem with this evaluation metric arises when the model does not assign some label to any of the dataset instances. In this case the model is not penalized during the computation of the micro-averaged precision, which only considers the relations where at least one sentence is correctly identified.

Regardless of these limitations, this metric is the one presented for all the models below, to be able to compare results with models proposed in the literature.

3 Random Forest

The first step consists of obtaining embeddings for the different tokens of each sentence. A downsized version of the pre-trained FastText model is used [Bojanowski et al. \[2016\]](#). This results in a more compact embedding of each token, specifically of dimension 100. This is done to optimize resources and accelerate processing times, all while retaining a meaningful representation of semantic and morphological information pertaining to words within our specific context.

3.1 Preparing data

The following steps are performed in order to collectively prepare the DataFrame with geometric, semantic, and categorical features for training a Random Forest classifier:

- The distance between subject and object is calculate using midpoints.
- The subject and object embeddings are obtained from their tokens using the pre-trained FastText model
- The cosine similarity between corresponding pairs of subject and object embeddings is computed and used as feature.
- Categorical features like 'subj-type' and 'obj-type' are one-hot encoded.
- Part-of-speech tags, under the column of stanford POS, are binary encoded.

3.2 Classification

A grid search is conducted to find the optimal configuration for the random forest. The search involves iterating over different combinations of hyperparameter values. For this problem the number of trees in the forest, and the maximum depth are tuned; the specific details can be found in the associated notebook. For each combination, a classifier is trained on a designated training dataset. The goal is to identify the hyperparameter values that result in the highest performance on a validation dataset. The performance metric used here is the F1 score described in [Section 2](#).

Throughout the grid search, the code keeps track of the best F1 score achieved and the corresponding set of hyperparameters. Once the optimal hyperparameters are identified, a final classifier is created using these values and trained on the entire training dataset. The performance of this optimized classifier is then evaluated on the test dataset.

The significant imbalance in the initial dataset, where the majority of instances are labeled as "no relation" among the 42 possible classes (relations), poses a potential challenge. The imbalance can lead to several issues as biased model and poor generalization. To address these issues, instances of different classes are weighted in a way that is inversely proportional to their frequency in the dataset. This helps by giving more importance to the minority class instances.

3.3 Results and error analysis

With the grid search procedure, the best hyper-parameters were identified to be 300 for the number of trees and 20 for the maximum depth. Table The following observations are derived by looking at the results in Tables [??](#), [??](#) :

Precision	Recall	F1
49.029%	34.165%	40.269%

Table 1: Precision, Recall and F1 Values on the test set

1. **org:alternate_names:** The model demonstrates a high precision of 73.53%, indicating accurate predictions. However, the recall is comparatively lower at 23.47%, suggesting that actual relationships of this type are often missed. The resulting F1 score is 35.59%.
2. **per:title:** This category exhibits precision (58.79%) and recall (85.60%), leading to an F1 score of 69.71%. This is the relation where the model achieves a higher F1 score.
3. **org:founded:** Perfect precision is achieved, but the recall is only 2.70%. While the model rarely makes mistakes in predicting this relation, it misses a substantial number of actual instances, resulting in a nuanced F1 score.
4. **per:employee_of:** The model struggles in this category with a precision of 31.29% and a recall of 61.74%. It tends to miss many instances while making a significant number of incorrect predictions.
5. **per:religion:** The model achieves reasonable precision (57.14%) but a relatively low recall of 8.51%, resulting in an F1 score of 14.81%. Identifying instances of individuals' religious affiliations proves to be challenging.

These detailed insights guide potential refinements to the model, such as targeted feature engineering, hyperparameter tuning, or additional data preprocessing. Addressing disparities in precision and recall across different relation types is crucial for enhancing the overall classifier performance.

Relation Category	Precision (%)	Recall (%)	F1 Score (%)
per:title	58.79	85.60	69.71
org:alternate_names	73.53	23.47	35.59
org:city_of_headquarters	70.21	40.24	51.16

Table 2: Best-Performing Relation Categories

Relation Category	Precision (%)	Recall (%)	F1 Score (%)
org:dissolved	100.00	0.00	0.00
per:religion	57.14	8.51	14.81

Table 3: Some Categories where model performs badly

Additionally, a closer examination of specific relations like *org:founded_by* and *org:member_of* reveals significant challenges. The *org:founded_by* relation is primarily misclassified as *org:top_members/employees* and *no_relation*. For *org:member_of*, 'no_relation' was the most frequent misclassification (14 instances), while 'org:country_of_headquarters' and 'org:parents'.

Relation	count
no_relation	16
org:top_members/employees	52

(a) Class. of *org:founded_by* relations

Relation	count
no_relation	14
org:country_of_headquarters	3
org:parents	1

(b) Class. of *org:member_of* relations

4 Deep Learning Methods

For what concerns the models in this section, each model is trained using 30 epochs and a batch size of 128 (due to computational restrictions).

Moreover, for each of the different models considered, the one achieving higher F1 score on the validation set is saved.

Finally, error analysis is performed only on the model achieving the highest F1 score on the validation set.

The best model is then saved and used for further analysis. Additional details for the different approaches are described below.

A vocabulary of length 3000 is built considering the most frequent words in the training set and an index is assigned to each of them.

For the models in Section 4, 4 we use the following approach to try and encode the position of the subject and the object.

Consider for example the following sentence from the training dataset:

"Tom Thabane resigned in October last year to form the All Basotho Convention ..."

with the subject being Tom Thabane and the object being the All Basotho Convention. We use additional special tokens <SUBJ> and <OBJ> to provide the model with information about the subject and the object. As such the sentence above is transformed into:

"<SUBJ> Tom Thabane <SUBJ> resigned in October last year to form the <OBJ> All Basotho Convention <OBJ> ...".

This should help with the fact that otherwise, the model would have no way to distinguish cases where the same sentence is used but the relation to be extracted is between different elements in the sentence.

Basic DL As first baseline model, a very basic Neural Network is considered. Each sentence in the dataset is converted into a vector of length equal to the vocabulary size using `CountVectorizer` by `scikit-learn`. No pre-processing is performed since the analysis of the task in the previous milestone pointed out potential problems in doing so. This vector is then given as input to the network.

It consists of a fully connected network with only one hidden layer with 200 hidden neurons. To move from these to the output layer, first a Dropout operation is performed, followed by the application of the ReLU activation function and a final linear operator. The output is then transformed using a softmax function, to ensure that it stays in range [0, 1].

To try and account for the large amount of imbalance between the classes, the loss function is weighted to penalise errors on smaller classes. A weighting factor which is inversely proportional to the number of occurrences of a certain class in the training set is used.

The main issue with this model is that it has no way to distinguish two data points where we have the same sentence but with different subjects and objects among which the relation needs to be classified. This means that particularly the unweighted model will be quite bad, and will end up

assigning label *no_relation* whenever the majority of instances from the same document have as correct label *no_relation*. The results in Table 5 show the model where the loss function is weighted

Model	Precision (micro)	Recall (micro)	F1 (micro)
DL1	17.694%	1.214%	2.272%
DL1 with weights	7.417%	25.294%	11.470%

Table 5: Results for Simple Neural Network on validation data

seems to be performing better, with a significant increase in recall, coming at a loss in precision. This is an aspect that is much dependent on the chosen metric, as was previously discussed in Section 2.

Embedding Layer For all models shown from now on, the starting point is a representation of each word in the sentences with the position of that word in the vocabulary. As mentioned before, the sentences with the additional Subject and Object tag are used. All words which are NOT included in the vocabulary are assigned to a special unknown word. Moreover, padding is required to place sentences in the tensors and a special padding word is added to the vocabulary; more details on this can be observed in the notebook.

The model presented here assigns to each word of the vocabulary an embedding (which is itself trained during the model training) of dimension 100. Afterwards, max pooling is applied to obtain from the embedding of all words a single vector representation for each sentence. On this, a simple linear layer is applied, followed by the usual softmax function. The results shown in Table 6 are

Model	Precision (micro)	Recall (micro)	F1 (micro)
DL2	24.593%	2.226%	4.082%
DL2 with weights	4.470%	18.396%	7.193%

Table 6: Results for Embedding matrix followed by linear layer on validation data

similar to those for the model without embeddings.

LSTM This model was explored more than the previous two, as LSTM used to be the gold standard for many NLP applications before the introduction of LLMs. For simplicity, the models were restricted to work with only 1 LSTM layer, while works such as the ones in Zhang et al. [2017] use more than one.

The standard structure of the network is briefly described below, and the code contains the implementation for both a uni-directional and a bi-directional LSTM layer. The input text is embedded in the same way as in Section , with training of the embeddings included. Afterwards, the embedding is passed through an LSTM layer. For the uni-directional case, the hidden vector representation of the last word in the sentence is extracted. For the bi-directional case, the hidden vector for the last words in both forward and backward passes are concatenated.

This vector is then passed through a linear layer and mapped into a hidden representation of dimension 64. Afterwards it is passed through a ReLU activation function followed by dropout to try and reduce overfitting during the training phase. Then it is mapped with another linear transformation to the output layer, upon which the softmax function is applied.

For both the uni and bi-directional case, different levels of dropout were tried. To be more specific,

Model Configuration	Precision (micro)	Recall (micro)	F1 (micro)
LSTM1 w.o. dropout	27.221%	6.990%	11.124%
LSTM1 w.o dropout and w. weights	2.151%	8.756%	3.453%
LSTM1 w. Dropout 0.1	34.312%	9.032%	14.300%
LSTM1 w. Dropout 0.25	25.996%	7.800%	11.999%
LSTM1 w. Dropout 0.50	39.170%	7.119%	12.049%
LSTM2 w.o. dropout	32.221%	10.302%	15.612%
LSTM2 w. Dropout 0.1	38.619%	10.081%	15.988%
LSTM2 w. Dropout 0.25	36.739%	12.104%	18.209%
LSTM2 w. Dropout 0.50	25.744%	4.617%	7.830%

Table 7: Results for uni and bi-directional LSTM on validation data

dropout probabilities of 0, 0.1, 0.25, 0.5 were considered, with 0 being the case of no dropout. Table 7 shows the results obtained for uni (LSTM1) and bi-directional LSTM models (LSTM2), with different options for dropout. The addition of weights similar to the ones used in the previous models was excluded after they led to much poorer results on the LSTM1 model (shown in the second row of the table).

It appears that the bi-directional LSTM model performs generally better than the uni-directional one. It performs particularly well with a dropout probability of 0.25. Model **LSTM2 w. Dropout 0.25** performs better than all other models considered so far. For this reason, it is chosen as the best out of those selected and further analyzed in Section 4.1.

4.1 Error Analysis

A step-by step error analysis of the model is described in the associated notebook. Some general insights derived from it are reported here.

First of all the model was evaluated on the unseen test dataset, and the results are reported in Table 8. The model performance is only slightly worse on the test dataset than on the validation

Model Configuration	Precision (micro)	Recall (micro)	F1 (micro)
LSTM2 with Dropout 0.25	34.156%	10.827%	16.442%

Table 8: Results for LSTM2 with Dropout 0.25 on test data

dataset, due to a small drop in both micro-averaged precision and recall. Due to the limitations of the evaluation metric described in Section 2, a more detailed assessment is required.

One major issue with this model that was spotted immediately, is that on the test data it only outputs 9 different relations when considering the relation classification task. Table 9 shows the precision and recall for the 6 instances where non-zero recall was achieved on the test dataset. This clearly shows that the model is not satisfactory, and the micro-averaged precision is boosted due to the limitations described in Section 2.

Another thing to note is that the model is unable to predict correctly even relations where there are a significant number of examples in the training set, and not only the rare ones.

Table 10 shows the 6 most frequent relations in the training dataset. However, relationship *per:employee* is the 4-th most common one and not a single instance is correctly identified in the test

Relation	Precision (%)	Recall (%)	F1 (%)	N of instances
org:alternate_names	39.58	35.68	37.53	213
org:top_members/employees	34.39	53.18	41.77	346
org:website	57.14	15.38	24.24	26
per:age	83.33	2.50	4.85	200
per:date_of_birth	28.57	22.22	25.00	9
per:title	30.38	17.80	22.45	500

Table 9: Precision, Recall, F1, and Counts for Relations with non-zero Recall on test dataset

Relation	#
no_relation	51367
per:title	2287
org:top_members/employees	1753
per:employee_of	1441
org:alternate_names	742
org:country_of_headquarters	438

Table 10: Counts for Various Relations

set. A similar thing occurs for *org:country_of_headquarters*, which is the 6-th most frequent one. Tables 11a and 11b show what labels are wrongly assigned to relations of these two types. In

Relation	count
no_relation	217
org:top_members/employees	25
per:title	18
org:alternate_names	4

(a) Class. of *per:employee_of* relations

Relation	count
no_relation	97
org:top_members/employees	6
org:alternate_names	5

(b) Class. of *org:country_of_headquarters* relations

both cases, we note that the main issue is the assignment of *no_relation* to both. For relation *per:employee_of* we also observe some wrong assignments to relations which are not completely unrelated, and this type of error would be more expected.

Further error analysis could be conducted for each of the other instances, but we believe that the main issue to address is the misclassification into *no_relation*.

5 Conclusions and Further Improvements

Given the current implementations, the random forest model seems to outperform the deep learning techniques proposed. One possible reason is that these latter were simple adaptations of models employed for other NLP tasks such as text classification. In fact, the only modification to better suit the task of relation extraction was to add the <SUBJ> and <OBJ> tags. However, while this allowed the model to distinguish in some cases the same sentence with different subject and object, by looking in more detail in section 4.1, the model still assigns many of the sentences to *no_relation*,

which is the most frequent one by a large amount in the training set. This is one of the key issues which will have to be addressed in the further developments of this project and while the RF model is slightly better in this regard, it still suffers from the issue.

Since the RF model performs better, this could for example be done by passing the additional variables which were not used in the DL models, such as the POS, NER and other features of the subject and the object. Another measure which was not used in the DL models was a measure of distance between the words in the sentences. This should be incorporated as the positional encoding of the different words with respect to the subject and object of the relation seem to be commonly used as in [Zhang et al. \[2017\]](#).

Another potential cause of the disparity between the two could be the fact that for the RF model, pre-trained embeddings were used, while these were trained on the training set for the DL models. Approaches which use pre-trained embeddings could be tried out.

As a final note, determining an appropriate architecture and appropriate hyper-parameters is trickier for DL models with this issue being complicated by the higher computational requirements for their training.

References

- Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. Enriching word vectors with subword information. *arXiv preprint arXiv:1607.04606*, 2016.
- Yuhao Zhang, Victor Zhong, Danqi Chen, Gabor Angeli, and Christopher D. Manning. Position-aware attention and supervised data improve slot filling. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 35–45, Copenhagen, Denmark, September 2017. Association for Computational Linguistics. doi: 10.18653/v1/D17-1004. URL <https://aclanthology.org/D17-1004>.